

Ibex back-end design flow

简介

此文件将向您介绍一套实现design ibex从RTL转变成GDS的flow。通过该文件的指导您将会掌握如何运行这一套flow以及对flow的拓展。

Design介绍

我们将会用130nm工艺库sky130hd实现design ibex从RTL到GDS的转变，ibex是一个开源的32 bit RISC-V CPU core (RV32IMC/EMC)，如需了解更多请参考https://ibex-core.readthedocs.io/en/latest/02_user/index.html

TCL知识简介

TCL 的全称是 Tool Command Language，它是由 UCA berkeley 开发的一种开放的工业标准语言，目前主流的EDA工具均支持使用Tcl命令指导工具行为。详细的Tcl语法知识请参考《Tcl/Tk入门经典(第2版)》

MakeFile知识简介

linux环境下，当用户编译文件过多的时候，使用makefile可以帮助模块化编译文件，makefile是一个脚本文件，根据规则，来执行相应的脚本文件，实现自动化编译。在大型IC设计中往往会使用MakeFile构建工程，指导脚本运行。详细知识请参考[概述 — 跟我一起写Makefile 1.0 文档 \(seisman.github.io\)](#)

环境准备

运行这套flow，您将需要准备以下工具，并将相关启动命令配置于~/.bashrc中

- Desxx Coxxx2019
- Forxxx2019
- Innxxx18
- Prixxx2019
- Calixxx2019
- Staxxx2019
- Volxxx18

Flow配置

这一部分将会展示flow包含的目录与脚本

```
|-- designs # 承载工艺库以及design相关配置文件
|   |-- sky130hd
|   |   |-- ibex # design配置件目录，包含了ibex.sdc,io.file等设计相关输入件
```

```
| | `-- pdk #工艺库目录
| |     |-- all_stdcell_netlist
| |     |-- all_stdcell_spice
| |     |-- gds
| |     |-- lef
| |     `-- lib
| `-- src #RTL文件目录
|     `-- ibex
|-- result #flow结果存放目录
`-- scripts #flow脚本存放目录
    |-- extractRC
    |-- fml
    |-- ir_v
    |-- lib2db
    |-- pr
    |-- pv
    |-- sta
    `-- syn
```

```
cd ibex_work_upload
```

- 工艺库相关的变量配置件

详细变量配置请查看运行说明部分

```
less ./designs/sky130hd/pdk/config.mk
```

- design相关的变量配置件

你需要在该配置件中指定design的RTL文件, SDC文件, IO_file

```
less ./designs/sky130hd/ibex/config.mk
```

```
export VERILOG_FILES = ./designs/src/$(DESIGN_NICKNAME)/ibex_alu.v \
    ./designs/src/$(DESIGN_NICKNAME)/ibex_branch_predict.v \
    ./designs/src/$(DESIGN_NICKNAME)/ibex_compressed_decoder.v \
    ./designs/src/$(DESIGN_NICKNAME)/ibex_controller.v \
    ./designs/src/$(DESIGN_NICKNAME)/ibex_core.v \
    ./designs/src/$(DESIGN_NICKNAME)/ibex_counter.v \
    ...
```

```
export PR_SDC_FILE =
    ./designs/$(PLATFORM)/$(DESIGN_NICKNAME)/constraint_for_pr.sdc
export SDC_FILE = ./designs/$(PLATFORM)/$(DESIGN_NICKNAME)/constraint.sdc
```

```
export IO_FILE = ./designs/$(PLATFORM)/$(DESIGN_NICKNAME)/io.file
```

- **scripts目录**用于承载运行flow相关工具的脚本, 包含以下目录和文件

syn	extractRC	pr	fml	sta	pv	static_ir
dc_main.tcl dc_report.tcl dc_setup.tcl	extractRC.cmd	init.tcl mmmc.view floor_plan.tcl place_io.tcl power_plan.tcl placement.tcl cts.tcl post_cts_opt.tcl routing.tcl routing_opt.tcl chip_done.tcl	fml_main.tcl	lib_setup.tcl run_timing.tcl	gds_merge.tcl create_text_ibex.tcl drc_rule lvs_rule trans_spi.sh	gen_techfile.tcl load_design.tcl run_static.tcl

- **result目录**用于承载flow运行过程产生的过程输出件或结果文件，这里用P&R过程的结果目录举例：

```
data log report
```

data目录用于存放产生的数据交换件或结果文件

log目录用于存放流程运行的log

report目录用于存放流程运行中产生的相关报告

- **Makefile**用于承载运行各个步骤的指令

```
less ./Makefile
```

实现目标

以下表格展示的ibex design将要实现的设计规格

Power	Timing	Area
TODO	17.4ns	570um*567um

运行说明

这一部分您将了解如何运行这一套flow，我们将以**步骤目标**，**运行命令**，**步骤说明**的形式向您说明每一个步骤，更详细的过程请参考脚本中的具体命令。

```
cd ibex_work_upload
```

Synthesis & Insert Scanchian

- 步骤目标
完成RTL2Netlist，并插入扫描链。检查setup时序结果
- 输入&输出

输入	输出
sky130_fd_sc_hd__tt_025C_1v80.db constraint.sdc ibex_core.v + ibex_counter.v + ibex_cs_registers.v + ... (ibex design相关所有verilog文件)	syn.log ibex_core.rpt.ddc ibex_core.svf ibex_core.syn.v check_design_after_syn.rpt check_timing.rpt dont_use.syn.tcl qor.rpt violation.rpt

- 运行命令

```
make syn
```

- 步骤说明

- 1.读入timing db
- 2.读入RTL
- 3.读入SDC
- 4.设置Path Group
- 5.设置dont_use cell
- 6.进行综合
- 7.插入scanchain
- 8.进行增量综合
- 9.输出网表和报告

步骤1.使用tcl指令如下

综合工具会使用[这里](#)链接到的工艺库中标准单元去映射电路网表

通常target_library为使用的标准单元库。link_library为所有可能使用到的库，包括使用到的ip。

```
set target_library $::env(DB_FILES)
set link_library "*" $target_library
```

Note:配置link_library时必须包含"*"，表示DC在引用实例化模块时，首先搜索已经调进memory中的模块和单元电路

步骤2.使用tcl指令如下

通常有两种方法读入verilog设计，read_file和analyze&elaborate, 这里采用后者。

```
elaborate $::env(DSIGN_NAME)
current_design $::env(DSIGN_NAME)
```

步骤3.使用tcl指令如下

```
source $::env(SDC_FILE)
```

该design是采用完全同步时钟进行设计以下，相关SDC命令如下

```
set clk_port [get_ports $clk_port_name]
create_clock -name $clk_name -period $clk_period $clk_port
```

input_delay与output_delay采取20%的时钟周期

```
set clk_io_pct 0.2
```

步骤4.使用命令如下

该design综合流程，我们需要重点关注寄存器与寄存器之间逻辑模块的时序优化，所以自定义了以下path_group，并设置相应的critical_range与weight。关于group_path命令更详细的解释您可以利用man group_path命令在DC工具中查看

```
reset_path_group -all
set reg [filter_collection [all_registers] "is_clock_gate != true"]
set input [all_inputs]
set output [all_outputs]
group_path -name reg2reg -weight 50 -critical_range 6 -from $reg -to $reg
group_path -name in2reg -weight 10 -critical_range 0.5 -from $input -to $reg
group_path -name reg2out -weight 10 -critical_range 0.5 -from $reg -to $output
```

步骤5.您可以使用pdk中的config.mk配置所需要禁用的stdcell

```
export DONT_USE_CELLS = \
    sky130_fd_sc_hd__probec_p_8 \
    sky130_fd_sc_hd__lpflow_bleeder_1 \
    sky130_fd_sc_hd__lpflow_clkbufkapwr_1 \
    sky130_fd_sc_hd__lpflow_clkbufkapwr_16 \
    sky130_fd_sc_hd__lpflow_clkbufkapwr_2 \
    sky130_fd_sc_hd__lpflow_clkbufkapwr_4 \
    ...
```

步骤6.使用以下命令执行综合

```
compile_ultra -timing_high_effort_script -scan
```

步骤7.使用以下命令完成对插入scanchain的设置

```
set_dft_insertion_configuration -preserve_design_name true
set_dft_signal -view existing_dft -type ScanClock -timing {45 55} -port {clk_i}
set_dft_signal -view existing_dft -port rst_ni -type Reset -active_state 0
set_dft_signal -view existing_dft -port test_en_i -type ScanEnable -active_state 1
set_dft_insertion_configuration -synthesis none -preserve_design_name true
set_autofix_configuration -type bidirectional -method input
create_test_protocol -infer_async -infer_clock
...
insert_dft
```

Note: 插入scanchain之后，我们可以利用dft_drc检查drc违例

步骤8. 进行增量综合过程

```
compile_ultra -scan -incremental
```

步骤9.利用以下命令输出综合报告，以及做P&R阶段我们所需要的门级网表和scan.def

```
redirect $::env(RESULT_DIR)/syn/report/check_design_after_syn.rpt {check_design}
redirect $::env(RESULT_DIR)/syn/report/qor.rpt {report_qor -nosplit}
redirect $::env(RESULT_DIR)/syn/report/violation.rpt {report_constraint -
all_violators}
write -format verilog -h -output
$::env(RESULT_DIR)/syn/data/$::env(DESIGN_NAME).syn.v
write -format ddc -h -output
$::env(RESULT_DIR)/syn/data/$::env(DESIGN_NAME).rpt.ddc
write_scan_def -output
$::env(RESULT_DIR)/scanchain/data/$::env(DESIGN_NAME).scan.def
```

Formal for Synthesis

- 步骤目标
综合之后的网表通过Formal验证
- 输入&输出

输入	输出
sky130_fd_sc_hd_tt_025C_1v80.db ibex_core.v + ibex_counter.v + ibex_cs_registers.v + ...(ibex design相关所有verilog文件) ibex_core.syn.v ibex_core.svf	unmatched_points.rpt matched_points.rpt fail_points.rpt unread.rpt aborted.rpt aborted.rpt rtl2syn.fss

- 运行命令
运行此命令之前，确保您已经成功执行 `make syn`

```
make fml_syn
```

- 步骤说明
 - 1.读入timing db
 - 2.读入svf
 - 3.读入reference design
 - 4读入implement design
 - 5.将scan cell的使能端无效
 - 6.match
 - 7.verify
 - 8.输出形式化验证相关报告

步骤1.使用以下命令

```
read_db -technology_library $::env(DB_FILES)
```

步骤2.读入.svf文件，此文件在综合过程中输出，记录了综合过程对网表的改变，在进行RTL级别与门级netlist的形式化验证时，此文件是必需的。

```
set_svf $::env(RESULT_DIR)/syn/data/$::env(DESIGN_NAME).svf
```

步骤3.使用以下命令读入未综合前的网表

```
foreach file $::env(VERILOG_FILES) {  
    read_verilog -r $file  
}  
set_top r:WORK/$::env(DESIGN_NAME)
```

步骤4.使用以下命令读入未综合后的网表

```
read_verilog -i $::env(RESULT_DIR)/syn/data/$::env(DESIGN_NAME).syn.v  
set_top i:WORK/$::env(DESIGN_NAME)
```

步骤5.您可以通过pdk中config.mk配置所需要禁用的scan cell的使能端

```
#Define the scan enable port  
export SCAN_ENABLE_PORT = SCE
```

步骤6.使用以下命令进行match并打印match相关报告

```
match  
report_matched_points > $::env(RESULT_DIR)/fml_syn/report/matched_points.rpt  
report_unmatched_points > $::env(RESULT_DIR)/fml_syn/report/unmatched_points.rpt  
report_unmatched_points -status unread >  
$::env(RESULT_DIR)/fml_syn/report/unread.rpt
```

步骤7.运行以下命令进行verify并打印verify相关报告

```
verify  
report_failing_points > $::env(RESULT_DIR)/fml_syn/report/fail_points.rpt  
report_aborted_points > $::env(RESULT_DIR)/fml_syn/report/aborted.rpt  
save_session -replace $::env(RESULT_DIR)/fml_syn/data/rtl2syn.fss
```

Note:.fss文件包含了所有读入formality工具的design,环境变量以及验证结果信息，您可以使用 `restore_session` 加载此文件

PR data init

- 步骤目标
完成进行PnR前的数据导入
- 输入&输出

输入	输出
ibex_core.syn.v sky130_fd_sc_hd_merged.lef sky130_fd_sc_hd.tlef sky130_fd_sc_hd__tt_025C_1v80.lib	init_design.enc init_design.enc.dat init.cmd init.log init.logv init_data_timing check_data_init.report

- 运行命令

运行此步骤前，请确保综合流程已经成功运行，并且通过形式化验证

```
make data_init
```

- 步骤说明

- 1.指定design的PG(power&groud) net名
- 2.指定综合之后的网表路径
- 3.指定top design名
- 4.指定lef文件
- 5.指定mmmc文件
- 6.初始化design
- 7.检查设计导入

步骤1.使用以下命令

```
set init_gnd_net {VSS}
set init_pwr_net {VDD}
```

步骤2.使用以下命令

```
set init_verilog $::env(RESULT_DIR)/syn/data/$::env(DESIGN_NAME).syn.v
```

步骤3.使用以下命令

```
set init_top_cell $::env(DESIGN_NAME)
```

步骤4.使用以下命令

```
set init_lef_file [split $::env(LEF_FILES)]
```

步骤5.采用以下脚本配置mmmc分析环境

创建RC Corner来提取Net的寄生电阻电容


```
create_rc_corner -name rc_max -preRoute_res 1.05 \  
    -cap_table $::env(CAP_TABLE) \  
    -preRoute_cap 1.05 \  
    -preRoute_clkres 1.05 \  
    -postRoute_res 1.05 \  
    -postRoute_cap 1.05 \  
    -postRoute_xcap 1.05 \  
    -postRoute_clkres 1.05 \  
    -postRoute_clkcap 1.05
```

创建setup time和hold time分析用的lib set

```
create_library_set -name lib_set_max -timing $::env(LIB_FILES)  
create_library_set -name lib_set_min -timing $::env(LIB_FILES)
```

创建Constrain Mode, 读入SDC

```
create_constraint_mode -name common -sdc_files $::env(PR_SDC_FILE)
```

使用library set和RC Corner创建Delay Corner

```
create_delay_corner -name delay_max -library_set {lib_set_max} -rc_corner  
{rc_max}  
create_delay_corner -name delay_min -library_set {lib_set_min} -rc_corner  
{rc_min}
```

使用Constraint mode和Delay corner创建analysis view

```
create_analysis_view -name max_view -constraint_mode {common} -delay_corner  
{delay_max}  
create_analysis_view -name min_view -constraint_mode {common} -delay_corner  
{delay_min}
```

指定setup time和hold time使用的analysis view

```
set_analysis_view -setup {max_view} -hold {min_view}
```

步骤6.使用以下命令

```
init_design
```

步骤7.使用以下命令检查设计导入

```
checkDesign -netList
```

Floorplan

- 步骤目标
完成芯片面积的设定与io的摆放
- 输入&输出

输入	输出
init_design.enc	floorplan.cmd floorplan.log floorplan.logv floor_plan.enc.dat floor_plan.enc

- 运行命令
运行此步骤前，请确保PR data init已经成功运行

```
make floorplan
```

- 步骤说明
 - 1.定义block的die area和core area
 - 2.摆放block port
 - 3.设置dont use cell
 - 4.保存当前设计

步骤1.通过pdk中config.mk配置定义block die area所使用的Cell Density和site

```
#-----  
# Floorplan  
# -----  
export PLACE_DENSITY = 0.4  
export PLACE_SITE = unithd
```

```
floorPlan -site $::env(SITE_NAME) -su 1 $::env(PLACE_DENSITY) 1 1 1 1
```

步骤2.通过读入io.file进行port摆放，或者利用命令editPin进行摆放

```
if {[info exists ::env(IO_FILE)]} {  
    if {$::env(IO_FILE)!=""} {  
        loadIoFile $::env(IO_FILE)  
    } else {  
        source $::env(SCRIPTS_DIR)/pr/place_io.tcl  
    }  
} else {  
    source $::env(SCRIPTS_DIR)/pr/place_io.tcl  
}
```

步骤3.通过以下命令设置dont use cell

```
foreach cell [split $::env(DONT_USE_CELLS)] {
    get_lib_cell */$cell
    set_dont_use [get_lib_cells */$cell] true
}
```

步骤4.使用以下命令保存design

```
saveDesign $::env(RESULT_DIR)/pr/data/floor_plan.enc
```

saveDesign 会在指定的目录下生成.enc和.enc.dat文件

可以在innovus工具中使用 `source $file_name.enc` 和 `restoreDesign $file_name.enc.dat` `$top_design_name` 加载当前设计

Powerplan

- 步骤目标
完成对设计的电源网络规划,并完成connectivity检查
- 输入&输出

输入	输出
floor_plan.enc	powerplan.enc powerplan.enc.dat power_plan.cmd power_plan.log power_plan.logv

- 运行命令
执行此步骤前, 请确保Floorplan步骤成功执行,并输出了floorplan.enc

```
make power_plan
```

- 步骤说明
 - 1.电源地逻辑连接
 - 2.创建P/G Stripe
 - 3.连接stdcell Follow pin
 - 4.检查电源地连接

步骤1.可以使用以下命令进行

相关电源地连接pin再pdk中config.mk中配置

```
globalNetConnect VDD -type pgpin -pin "$::env(PWR_PORT)" -inst *
globalNetConnect VDD -type tiehi -pin "$::env(PWR_PORT)" -inst *
globalNetConnect VDD -type net -net VDD
globalNetConnect VSS -type pgpin -pin "$::env(GND_PORT)" -inst *
globalNetConnect VSS -type tielo -pin "$::env(GND_PORT)" -inst *
globalNetConnect VSS -type net -net VSS
```

步骤2.使用以下命令进行

```
addStripe -nets {VSS VDD} \  
  -layer $::env(V_STRIPE_METAL) \  
  -direction vertical \  
  -width $::env(STRIPE_WIDTH) \  
  -spacing $::env(STRIPE_SPACING) \  
  -set_to_set_distance $::env(STRIPE_DISTANCE) \  
  -start_from left \  
  -start_offset 1 \  
  -uda power_stripe_v  
  
addStripe -nets {VSS VDD} \  
  -layer $::env(H_STRIPE_METAL) \  
  -direction horizontal \  
  -width $::env(STRIPE_WIDTH) \  
  -spacing $::env(STRIPE_SPACING) \  
  -set_to_set_distance $::env(STRIPE_DISTANCE) \  
  -start_from bottom \  
  -start_offset 1 \  
  -uda power_stripe_h
```

水平和垂直的Stripe金属可以在pdk中的config.mk进行

```
#Vertical stripe metal  
export V_STRIPE_METAL = met4  
#Horizontal stripe metal  
export H_STRIPE_METAL = met5
```

步骤3.可以使用pdk中config.mk配置sroute所有的metal

```
export SROUTE_MIN_LAYER = li1(1)  
export SROUTE_MAX_LAYER = met4(4)
```

步骤4.使用命令 `verifyConnectivity -type special -noAntenna -noUnroutedNet -`

`noWeakConnect` 检查结果

```
***** Start: VERIFY CONNECTIVITY *****  
Start Time: Mon Nov 21 11:23:44 2022  
  
Design Name: ibex_core  
Database Units: 1000  
Design Boundary: (0.0000, 0.0000) (556.1400, 551.4800)  
Error Limit = 1000; Warning Limit = 50  
Check specified nets  
Net boot_addr_i[0]: Found a geometry with bounding box (0.00,-0.00) (0.14,0.49) outside the design  
boundary.  
Violations for such geometries will be reported.  
**** 11:23:44 **** Processed 5000 nets.  
**** 11:23:44 **** Processed 10000 nets.  
  
Begin Summary  
  Found no problems or warnings.  
End Summary  
  
End Time: Mon Nov 21 11:23:44 2022  
Time Elapsed: 0:00:00.0  
  
***** End: VERIFY CONNECTIVITY *****  
Verification Complete : 0 Viols. 0 Wrngs.  
(CPU Time: 0:00:00.1 MEM: 0.000M)
```

Placement

- 步骤目标
完成stadcell的placement,Congestion情况合理,并输出了powerplan.enc
- 输入&输出

输入	输出
floor_plan.enc	powerplan.enc powerplan.enc.dat power_plan.cmd power_plan.log power_plan.logv placement_timing

- 运行命令
运行此步骤前，确保Powerplan步骤成功执行

```
make placement
```

- 步骤说明
 - 1.读入scan.def
 - 2.设置path group
 - 3.设置Place Mode
 - 4.设置Global routing Mode
 - 5.进行Place
 - 6.输出Place后的Congestion和Timing 报告

步骤1.读入在综合步骤输出的scan.def

```
defIn $::env(RESULT_DIR)/scanchain/data/ibex_core.scan.def
```

在做完Placement步骤后可以打开gui界面，在gui界面依次选择Place -> Display -> Scan Chain 选中Selected Scan Group, 选中Scan Chain名字，点击Display即可查看插链的情况

步骤2.设置group_path，并设置相应优化的权重

```
group_path -name reg2reg -from $reg -to $reg  
...  
setPathGroupOptions reg2cg -effortLevel high
```

步骤3.设置布局引擎的模式

```
setPlaceMode -reset  
setPlaceMode -place_global_ignore_scan true  
setPlaceMode -place_global_reorder_scan false  
setPlaceMode -place_global_place_io_pins false  
setPlaceMode -place_detail_legalization_inst_gap 2
```

步骤4.使用pdk中config.mk配置Global routing所使用的metal

```
#-----
# Place
# -----
export MIN_GLOBAL_ROUTE_LAYER = 2
export MAX_GLOBAL_ROUTE_LAYER = 5
```

步骤5.通过命令 `reportCongestion -overflow` 查看拥塞数据

Note: Congestion overflow 值一般小于1.5是对之后的绕线步骤比较友好的数值

```
innovus 3> reportCongestion -overflow
Usage: (36.7%H 37.8%V) = (3.813e+05um 3.073e+05um) = (140170 112994)
Overflow: 994 = 329 (0.79% H) + 665 (1.61% V)
```

Congestion distribution:

Remain	cntH		cntV	

-5:	3	0.01%	0	0.00%
-4:	9	0.02%	3	0.01%
-3:	23	0.06%	13	0.03%
-2:	68	0.16%	93	0.22%
-1:	167	0.40%	505	1.22%

0:	369	0.89%	1715	4.14%
1:	935	2.26%	3673	8.87%
2:	1894	4.57%	5324	12.86%
3:	3459	8.35%	5618	13.57%
4:	5215	12.59%	5326	12.86%
5:	29270	70.68%	19142	46.22%

CTS

- 运行命令

运行此步骤前，需要确保placement步骤成功执行，并输出了placement.enc

```
make cts
```

- 输入&输出

输入	输出
placement.enc	cts.enc cts.enc.dat cts.cmd cts.log cts.logv cts_timing

- 步骤目标

完成时钟树的综合

- 步骤说明

- 1.设置ccopt时用到的stdcell
- 2.将步骤一相关的stdcell dont use 状态设置为false

3.创建CTS的routing non-default rule

4.建立时钟树

5.进行ccopt

6.进行post cts后的优化

步骤1.和步骤2.您可以使用pdk中config.mk配置CTS所用到的stdcell

```
export CTS_BUF_CELL = sky130_fd_sc_hd__clbuf_4
export CTS_INV_CELL = sky130_fd_sc_hd__lpflow_clkinvkapwr_1 \
                      sky130_fd_sc_hd__lpflow_clkinvkapwr_2 \
                      sky130_fd_sc_hd__lpflow_clkinvkapwr_4 \
                      sky130_fd_sc_hd__lpflow_clkinvkapwr_8 \
                      sky130_fd_sc_hd__lpflow_clkinvkapwr_16
```

步骤3.您可以使用pdk中config.mk配置设置non-default rule的layer,spacing,width

```
#Set the routing non-default rule which are used for clk tree routing
export CTS_ROUTING_MUL = 2
export NDR_CTS_MIN_LAYER = met2
export NDR_CTS_MAX_LAYER = met4
```

```
#Set the routing non-default rule which are used for clk tree routing
add_ndr -name cts_1 \
  -width_multiplier "$ndr_min_layer:$ndr_max_layer $mul" \
  -spacing_multiplier "$ndr_min_layer:$ndr_max_layer $mul"
```

步骤4.您可以使用pdk中config.mk配置设置NanoRoute可以使用的metal范围

```
#Set the routing metal which are used for clk tree routing
export CTS_ROUTING_LAYER_RANGE = 6 2
```

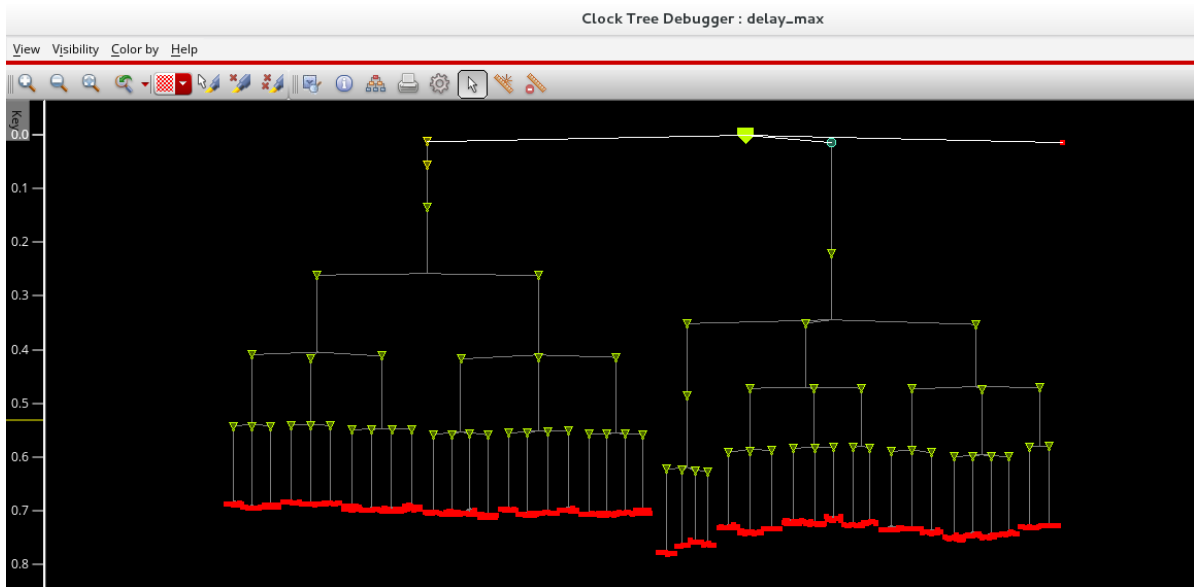
使用以下命令创建时钟树

```
create_ccopt_clock_tree_spec -file $::env(RESULT_DIR)/pr/data/clk.spec
source $::env(RESULT_DIR)/pr/data/clk.spec
```

步骤5.使用以下命令进行时钟树综合后的优化

```
ccopt_design -cts
```

使用gui界面的菜单栏打开Clock -> Clock Tree Debugger查看时钟树生成是否平整



使用命令 `report_ccopt_skew_groups` 查看时钟树的skew report

步骤6.执行时钟树综合后的Design优化

```
make post_cts_opt
```

优化后的timing报告和DRV

timeDesign Summary

Setup views included:

max_view

Setup mode	all	reg2reg	in2reg	reg2out	default	feedthr
WNS (ns):	0.029	0.029	1.241	1.115	8.317	4.070
TNS (ns):	0.000	0.000	0.000	0.000	0.000	0.000
Violating Paths:	0	0	0	0	0	0
All Paths:	3708	1924	3465	100	1	32

DRVs	Real		Total
	Nr nets(terms)	Worst Vio	Nr nets(terms)
max_cap	0 (0)	0.000	0 (0)
max_tran	0 (0)	0.000	0 (0)
max_fanout	0 (0)	0	0 (0)
max_length	0 (0)	0	0 (0)

Routing

- 运行命令

运行此步骤前, 请确保post_cts_opt成功执行, 并输出了post_cts_opt.enc

```
make routing
```

- 输入&输出

输入	输出
post_cts_opt.enc	routing.enc routing.enc.dat routing.cmd routing.log routing.logv routing_timing routing_opt_timing

- 步骤目标
完成block的routing，时序满足要求，完成电源地的检查和drc的检查
- 步骤说明
 - 1.设置NanoRouteMode
 - 2.进行Routing
 - 3.输出Routing后的时序报告
 - 4.进行Routing后的优化
 - 5.检查电源地连接
 - 6.检查drc违例

步骤1.您可以使用pdk中config.mk设置NanoRoute所使用的metal

```
# -----
# Route
# -----
export MIN_ROUTING_LAYER = 2
export MAX_ROUTING_LAYER = 6
```

使用以下命令设置Route引擎的模式

```
setNanoRouteMode -quiet -routeWithTimingDriven true
setAnalysisMode -analysisType onChipVariation
setNanoRouteMode -quiet -drouteEndIteration 70
setNanoRouteMode -quiet -drouteFixAntenna true
setNanoRouteMode -quiet -drouteUseMultiCutViaEffort medium
setNanoRouteMode -quiet -drouteMinSlackForWireOptimization 0.1
setNanoRouteMode -quiet -routeTopRoutingLayer $::env(MAX_ROUTING_LAYER)
setNanoRouteMode -quiet -routeBottomRoutingLayer $::env(MIN_ROUTING_LAYER)
#setNanoRouteMode -quiet -routeWithSiDriven true
setDelayCalMode -engine default -siAware true
```

步骤2.使用以下命令

```
routeDesign -globalDetail
```

步骤3.使用命令 `timeDesign -postRoute` 检查时序结果与DRV

Setup views included:
max_view

Setup mode	all	reg2reg	in2reg	reg2out	default	feedthr
WNS (ns):	-2.542	-2.542	0.825	-1.392	9.018	2.390
TNS (ns):	-1737.1	-1733.8	0.000	-3.316	0.000	0.000
Violating Paths:	1326	1320	0	6	0	0
All Paths:	3708	1924	3465	100	1	32

DRVs	Real		Total
	Nr nets(terms)	Worst Vio	Nr nets(terms)
max_cap	13 (13)	-0.045	13 (13)
max_tran	13 (90)	-0.427	13 (90)
max_fanout	0 (0)	0	0 (0)
max_length	0 (0)	0	0 (0)

步骤4.执行routing之后的优化步骤

make routing_opt

Setup views included:
max_view

Setup mode	all	reg2reg	in2reg	reg2out	default	feedthr
WNS (ns):	0.163	0.163	0.848	0.296	9.018	2.413
TNS (ns):	0.000	0.000	0.000	0.000	0.000	0.000
Violating Paths:	0	0	0	0	0	0
All Paths:	3708	1924	3465	100	1	32

DRVs	Real		Total
	Nr nets(terms)	Worst Vio	Nr nets(terms)
max_cap	0 (0)	0.000	0 (0)
max_tran	0 (0)	0.000	0 (0)
max_fanout	0 (0)	0	0 (0)
max_length	0 (0)	0	0 (0)

Density: 41.604%
Total number of glitch violations: 8

步骤5.使用命令 verifyConnectivity -type all 检查电源地连接

```

innovus 17> verifyConnectivity -type all
VERIFY_CONNECTIVITY use new engine.

***** Start: VERIFY CONNECTIVITY *****
Start Time: Mon Nov 21 13:19:44 2022

Design Name: ibex_core
Database Units: 1000
Design Boundary: (0.0000, 0.0000) (556.1400, 551.4800)
Error Limit = 1000; Warning Limit = 50
Check all nets
**** 13:19:44 **** Processed 5000 nets.
**** 13:19:44 **** Processed 10000 nets.
Net VDD: has an unconnected terminal.
**WARN: (IMPVFC-3): Verify Connectivity stopped: Number of errors exceeds the limit 1000
Type 'man IMPVFC-3' for more detail.

Begin Summary
    1000 Problem(s) (IMPVFC-96): Terminal(s) are not connected.
    1000 total info(s) created.
End Summary

End Time: Mon Nov 21 13:19:44 2022
Time Elapsed: 0:00:00.0

***** End: VERIFY CONNECTIVITY *****
Verification Complete : 1000 Viols.  0 Wrngs.
(CPU Time: 0:00:00.5  MEM: 0.000M)

```

步骤6.使用命令 `verify_drc` 检查DRC

```

VERIFY DRC ..... Sub-Area : 9 complete 0 Viols.
VERIFY DRC ..... Sub-Area: {141.120 277.760 282.240 416.640} 10 of 16
VERIFY DRC ..... Sub-Area : 10 complete 0 Viols.
VERIFY DRC ..... Sub-Area: {282.240 277.760 423.360 416.640} 11 of 16
VERIFY DRC ..... Sub-Area : 11 complete 0 Viols.
VERIFY DRC ..... Sub-Area: {423.360 277.760 556.140 416.640} 12 of 16
VERIFY DRC ..... Sub-Area : 12 complete 0 Viols.
VERIFY DRC ..... Sub-Area: {0.000 416.640 141.120 551.480} 13 of 16
VERIFY DRC ..... Sub-Area : 13 complete 0 Viols.
VERIFY DRC ..... Sub-Area: {141.120 416.640 282.240 551.480} 14 of 16
VERIFY DRC ..... Sub-Area : 14 complete 0 Viols.
VERIFY DRC ..... Sub-Area: {282.240 416.640 423.360 551.480} 15 of 16
VERIFY DRC ..... Sub-Area : 15 complete 0 Viols.
VERIFY DRC ..... Sub-Area: {423.360 416.640 556.140 551.480} 16 of 16
VERIFY DRC ..... Sub-Area : 16 complete 0 Viols.

Verification Complete : 0 Viols.

*** End Verify DRC (CPU: 0:00:08.8  ELAPSED TIME: 3.00  MEM: 0.0M) ***

```

Chip Finish

- 运行命令

执行该步骤时，请确保routing_opt步骤成功执行，并输出了routing_opt.enc

```
make chip_done
```

- 输入&输出

输入	输出
routing_opt.enc gds.map	chip_done.cmd chip_done.log chip_done.logv chip_done.enc.dat chip_done.enc ibex_core.gds ibex_lvs.vg ibex_routing.vg ibex_routing.def create_text_ibex.tcl hcell_list VDD.pp VSS.pp

- 步骤目标

输出设计的.def,.v,.gdsii文件

- 步骤说明

- 1.处理routing后的网表，删除空的Module与悬空的net
- 2.输出routing后的def
- 3.输出routing后的网表
- 4.输出lvs所用标记PG net text的脚本和static ir所用的P/G location脚本
- 5.输出lvs 和 static ir所用hcell list
- 6.输出GDS

步骤1.使用以下命令执行

```
deleteDanglingNet
deleteEmptyModule
```

步骤2.使用以下命令执行

```
set lefDefOutVersion 5.8
defOut -floorplan -netlist -routing $::env(RESULT_DIR)/pr/data/ibex_routing.def
```

步骤3.使用以下命令执行

```
saveNetlist $::env(RESULT_DIR)/pr/data/ibex_routing.vg
saveNetlist -excludeLeafCell -includePowerGround -flattenBus
$::env(RESULT_DIR)/pr/data/ibex_lvs.vg
```

Note: 在保存lvs所有的网表时，如果存在物理标准单元中有金属层的，需要使用到 `-includePhysicalCell` 包含进去

步骤4.在pdk中的config.mk指定使用的metal stack num

```
export LAYER_TEXT_NUM = 72
```

Note: 由于innovus在写出gds时不会标记VDD和VSS的位置，我们在做lvs时会找不到对应的net，所以我们需要写出一个给gds标记VDD和VSS的脚本，使用calibredrv工具将gds对应VDD和VSS net的位置打上label

步骤5.使用以下脚本写出

```
set hcell_file [open $::env(SCRIPITS_DIR)/pv/hcell_list w]
set hcell_list_ir [open $::env(SCRIPITS_DIR)/ir_v/cell_list w]
set cells [dbGet [dbGet top.insts.isPhysOnly 0 -p].cell.name]
set cells [lsort -u $cells]
foreach cell $cells {
    puts $hcell_file "$cell $cell"
    puts $hcell_list_ir "$cell"
}
close $hcell_file
close $hcell_list_ir
```

步骤6.在pdk中的config.mk指定使用的gds_map File

```
export GDS_MAP_FILE = $(DESIGN_HOME)/sky130hd/pdk/gds/gds.map
```

```
setStreamOutMode -textSize 5
setStreamOutMode -virtualConnection true
setStreamOutMode -uniquifyCellNamesPrefix true
setStreamOutMode -check_map_file true
streamOut $::env(RESULT_DIR)/pr_out/data/ibex_core.gds -mapFile
$::env(GDS_MAP_FILE) -libName DesignLib -units 1000 -mode ALL
```

Extract RC

- 运行命令

```
extract_rc
```

- 输入&输出

输入	输出
ibex_routing.def .nxtgrd .map sky130_fd_sc_hd.tlef sky130_fd_sc_hd_merged.lef	ibex.spef.gz

- 步骤目标

利用StarRC抽取routing之后网表spef

- 步骤说明

- 指定routing之后的def文件
- 指定lef文件
- 指定mapfile文件

- 4.指定nxtgrd文件
- 5.指定抽取spef时相关变量
- 以上步骤配置需要在 ./scripts/extractRC/extractRC.cmd 中配置

STA

- 运行命令

```
make run_pt
```

- 输入&输出

输入	输出
ibex_routing.vg constraint.sdc sky130_fd_sc_hd__tt_025C_1v80.db ibex.spef.gz	parasitics_command.log pt_shell_command.log run_pt.log ibex_core_session ibex_core_analysis_coverage.rpt ibex_core_check_timing.report ibex_core_report_constratints.report ibex_core_report_design.rpt ibex_core_report_net.rpt ibex_core_report_timing.report

- 步骤目标
- 利用prime time对routing之后网表进行时序检查
- 步骤说明

- 1.设置pt工具的相关变量
- 2.读入timing db
- 3.读入routing之后的网表
- 4.读入SDC
- 5.读入spef
- 6.输出反标报告
- 7.设置path group
- 8.输出Timing报告

步骤1.使用以下命令

```
set read_parasitics_load_locations true
set sh_source_uses_search_path true
set sh_command_log_file $::env(RESULT_DIR)/sta/log/pt_shell_command.log
set parasitics_log_file $::env(RESULT_DIR)/sta/log/parasitics_command.log
printvar sh_eco_enabled
set EnablePTSI true
if {$EnablePTSI == "true"} {
    set si_enable_analysis true
    set si_xtalk_double_switching_mode clock_network
}
```

步骤2.读入timing lib

```
source $::env(SCRIPTS_DIR)/sta/lib_setup.tc
```

步骤3.读入PnR输出的网表

```
read_verilog $::env(RESULT_DIR)/pr_out/data/ibex.vg
set DESIGN_NAME ibex_core
current_design $DESIGN_NAME
link
```

步骤4.读入sdc

```
source -e -v $::env(SDC_FILE)
```

步骤5.读入spef

```
read_parasitics -keep_capacitive_coupling -format spef
$::env(DESIGN_HOME)/sky130hd/ibex/ibex_mine.spef
```

步骤6.输出反标报告

```
redirect $::env(RESULT_DIR)/sta/report/ibex_core_annotated_parasitics.report
{report_annotated_parasitics}
```

Note: 注意检查Pin to pin net Not Annotated的数值是否为0，当不为0时，说明RC反标存在问题

步骤7.设置path group

```
set reg [filter_collection [all_registers] "is_integrated_clock_gating_cell !=
true"]
set input [all_inputs]
set output [all_outputs]
set ckgating [filter_collection [all_registers] "is_integrated_clock_gating_cell
== true"]
set ignore_path_groups [list inp2reg reg2out reg2out feedthr]
#set mem [get_cells -q -hier -filter "@is_hierarchical == false && @is_macro_cell
== true"]

group_path -name reg2reg -from $reg -to $reg
group_path -name reg2cg -from $reg -to $ckgating
group_path -name in2reg -from $input
group_path -name reg2out -to $output
```

步骤8.输出timing报告

```
redirect $::env(RESULT_DIR)/sta/report/ibex_core_report_constraints.report
{report_constraint -significant_digit 4 -all_violators -nosplit}
...
```

DRC

- 运行命令

```
make drc
```

- 输入&输出

输入	输出
ibex_core.gds sky130_fd_sc_hd.gds drc_rule	ibex_core.merge.gds drc.log merge.log DRC_RES.db

- 步骤目标

利用calibre对输出的版图进行drc

- 步骤说明

1.merge stdcell和top design的gds

2.执行drc验证

3.输出result db

4.读入result并查看违例点

步骤1.使用以下命令执行

因为innovus吃进去只是lef文件，是一个简化模型，并没有base layer的定义。那么写出来的gds并不是个完整的gds文件，所以我们需要将stdcell的gds文件与innovus吐出来的gds进行merge，才能带有完整的版图信息，才能进行接下来drc和lvs

```
#gds_merge.tcl
layout filemerge -in $::env(RESULT_DIR)/pr/data/$::env(DESIGN_NAME).gds \
-in $::env(DESIGN_HOME)/sky130hd/pdk/gds/sky130_fd_sc_hd.gds \
-topcell ibex_core \
-mode overwrite \
-out $::env(RESULT_DIR)/pv/drc/data/$::env(DESIGN_NAME).merge.gds
```

```
calibredrv $(SCRIPTS_DIR)/pv/gds_merge.tcl | tee
$(RESULT_DIR)/pv/drc/log/merge.log
```

步骤2.使用以下命令执行

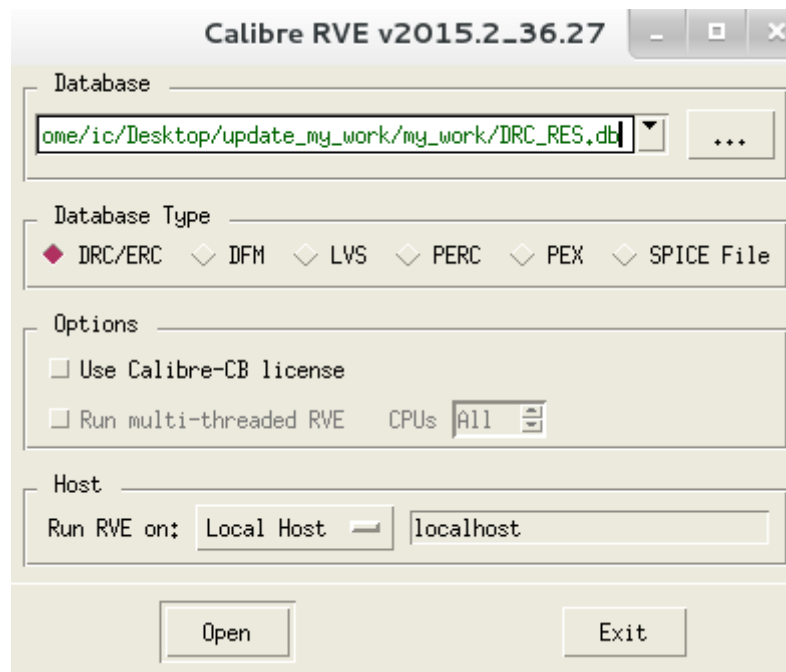
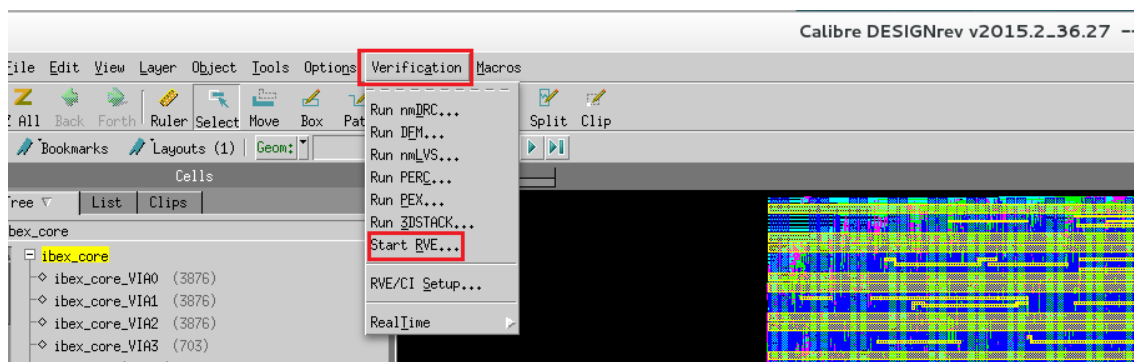
```
calibre -hier -drc $(SCRIPTS_DIR)/pv/drc_rule | tee
$(RESULT_DIR)/pv/drc/log/drc.log
```

步骤3.使用以下命令执行

```
calibre -hier -drc $(SCRIPTS_DIR)/pv/drc_rule | tee
$(RESULT_DIR)/pv/drc/log/drc.log
```

步骤4.使用 `calibredrv -m ./result/pv/drc/data/ibex_core.merge.gds` 打开layout

使用gui界面加载进DRC结果db



LVS

- 运行命令

```
make lvs
```

- 输入&输出

输入	输出
ibex_core.merge.gds .cdl hcell_list lvs_rule stdcell.v	

- 步骤目标

利用calibre对输出的版图进行lvs

- 步骤说明

- 1.将输出的gds打上P/G net的text
- 2.将routing后的网表转化成.spi网表
- 3.将gds转成.spi网表

4.进行Lvs验证

步骤1.将输出的gds打上teminate的text

```
calibredrv -64 $(SCRIPTS_DIR)/pv/create_text_ibex.tcl
```

步骤2.将ibex_lvs.vg网表转化成.spi网表

```
bash $(SCRIPTS_DIR)/pv/trans_spi.sh
```

使用v2lvs命令时, 需要指定stdcell的spice文件和网表文件, 您可以在pdk中的config.mk进行配置

```
# -----  
# Lvs  
# -----  
export STDCELL_SPICE =  
export STDCELL_NETLIST =
```

```
v2lvs -s1 \  
-s $cells_path/all_stdcell_spice/sky130_fd_sc_hd__einvn_0.cdl \  
-s $cells_path/all_stdcell_spice/sky130_fd_sc_hd__einvn_8.cdl \  
-s $cells_path/all_stdcell_spice/sky130_fd_sc_hd__einvn_1.cdl \  
-l $cells_path/all_stdcell_netlist/sky130_fd_sc_hd__einvn_4.v.copy \  
...
```

步骤3.在lvs rule中指定需要转化的gds文件

```
LAYOUT_PATH "./result/pv/lvs/data/ibex_core.text.gds"
```

步骤4.使用以下命令

```
calibre -lvs \  
    -hier \  
    -hcell $(SCRIPTS_DIR)/pv/hcell_list $(SCRIPTS_DIR)/pv/lvs_rule \  
    | tee $(RESULT_DIR)/pv/lvs/log/lvs.log
```

Power analysis

- 运行命令

```
make static_ir
```

- 输入&输出

输入	输出
qrcTechfile chip_done.enc.dat ibex.spef.gz VDD.pp VSS.pp	staticPower.db static.rpt static_VSS.ptisavg static_VDD.ptisavg tech_pgv VDD_85C_avg_1

- 步骤目标
利用Voltus完成对design的静态功耗与静态ir drop的分析

- 步骤说明
 - 1.读入routing之后的保存的.enc.dat
 - 2.生成PGV
 - 3.读入spef
 - 4.设置Static power analysis mode
 - 5.进行Static power analysis
 - 6.设置Static Rail analysis mode
 - 7.设置rail analysis mode
 - 8.读入电源地location文件
 - 9.读入static power analysis输出的.ptiavg文件
 - 10.进行static rail分析

步骤1.使用以下命令

```
read_design -physical_data $::env(RESULT_DIR)/pr/data/chip_done.enc.dat
$::env(DESIGN_NAME)
```

步骤2.需要输入qrcTechFile,您可以通过pdk中的config.mk进行设置

```
# -----
# IR Drop
# -----
export QRC_FILE =
```

并使用以下命令生成tech_pgv

```
set_pg_library_mode \
  -celltype          techonly \
  -ground_pins       VSS \
  -power_pins        "VDD $::env(PWR_NETS_VOLTAGES)" \
  -default_area_cap   0.01 \
  -extraction_tech_file "$::env(QRC_FILE)"
#-decap_cells        "$decap_cell_name" \
#-filler_cells        "$filler_name" \
#-cell_decap_file     ../data/voltus/decap.cmd

generate_pg_library \
  -output             ./result/ir_v/data/tech_pgv
```

步骤3.使用以下命令

```
spefIn -rc_corner rc_max $::env(RESULT_DIR)/extractRC/data/ibex.spef.gz
```

步骤4.使用以下命令

```
report_power \  
-outfile static.rpt
```

步骤5.可以启动Power Debug查看Static Power analysis result db

```
start_gui  
read_power_rail_results -power_db  
result/ir_v/data/staticPower.db
```

步骤6.您可以通过pdk中的config.mk设置电源地的电压值和阈值电压，以及温度条件

```
export PWR_NETS_VOLTAGES = 0.9  
export PWR_THRESHOLD = 0.85  
export GND_NETS_VOLTAGES = 0.0  
export GND_THRESHOLD = 0.05  
export RAIL_ANALYSIS_TEMPERATURE = 85
```

步骤7.使用以下命令执行

```
set_rail_analysis_mode \  
-method static \  
-accuracy xd \  
-analysis_view max_view \  
-power_grid_library { \  
    ./result/ir_v/data/tech_pgv/techonly.cl \  
} \  
-enable_rlrp_analysis true \  
-verbosity true \  
-temperature "$::env(RAIL_ANALYSIS_TEMPERATURE)"
```

Note:这里的techonly.cl是进行power analysis时吐出来的

步骤8.使用以下命令

```
set_power_pads \  
-reset  
  
set_power_pads \  
-net VDD \  
-format xy \  
-file ./scripts/ir_v/VDD.pp  
  
set_power_pads \  
-net VSS \  
-format xy \  
-file ./scripts/ir_v/VSS.pp
```

步骤9.使用以下命令

```
set_power_data -reset
set_power_data \
  -format                current \
  { \
    ./result/ir_v/data/static_VDD.ptiavg \
    ./result/ir_v/data/static_VSS.ptiavg \
  }
```

步骤10.启动static rail analysis

```
analyze_rail \
  -output          ./result/ir_v/report \
  -type            net \
                  VDD
```

NOTE:由于这里我们design没有规划电压域，所以选择分析的模式(-type)为net

您可以启动Power Debug查看Static rail analysis plot

```
read_power_rail_results -power_db \
result/ir_v/data/staticPower.db \
-rail_file
result/ir_v/report/VDD_85C_avg_1 \
```